

Taller Linux — Automatización con Ansible

Proyecto de automatización con Ansible para el despliegue de una aplicación web PHP con base de datos MariaDB, sobre servidores Ubuntu (base de datos) y CentOS (servidor web).

Arquitectura

- **ubuntu-db:** servidor Ubuntu con MariaDB, base de datos `cumple`.
- **centos01** (grupo `centos`): servidor CentOS Stream con Apache, PHP y la aplicación web que consulta la base de datos.

1. Requisitos previos

- Ansible instalado en el nodo de control (`ansible-core >= 2.14` recomendado).
- Acceso SSH configurado a ambos servidores, con un usuario que tenga privilegios de `sudo`.
- Python instalado en los hosts remotos (requisito estándar de Ansible).

2. Instalación de colecciones

Este proyecto depende de las siguientes colecciones de Ansible. Instalarlas antes de correr cualquier playbook:

```
bash
```

```
ansible-galaxy collection install community.mysql
ansible-galaxy collection install community.general
ansible-galaxy collection install ansible.posix
```

O todas juntas en un solo comando:

```
bash
```

```
ansible-galaxy collection install community.mysql community.general ansible.pos
```

Verificar la instalación:

```
bash
```

```
ansible-galaxy collection list
```

Deberían aparecer listadas `community.mysql`, `community.general` y `ansible.posix`.

Si el proyecto usa un archivo `requirements.yml`, alternativamente se pueden instalar todas de una:

yaml

```
# requirements.yml
collections:
  - name: community.mysql
  - name: community.general
  - name: ansible.posix
```

bash

```
ansible-galaxy collection install -r requirements.yml
```

3. Inventario

Archivo `inventory` (o `hosts.ini`) con los grupos y credenciales de conexión SSH a cada servidor.

ini

```
[database]
ubuntu-db ansible_host=<IP_DEL_SERVIDOR_DB> ansible_user=sysadmin ansible_ssh_p

[centos]
centos01 ansible_host=<IP_DEL_SERVIDOR_WEB> ansible_user=sysadmin ansible_ssh_p
```

Nota de seguridad: las credenciales de arriba son de ejemplo. En un entorno real, evitar guardar contraseñas en texto plano en el inventario — usar autenticación por clave SSH (`ansible_ssh_private_key_file`) y/o encriptar las variables sensibles con **Ansible Vault**:

Vault:

bash

```
ansible-vault encrypt_string 'MiPasswordSSH' --name 'ansible_ssh_pass'
```

Y ejecutar los playbooks con `--ask-vault-pass` o `--vault-password-file`.

Variables de la base de datos (`vars/database.yml`)

yaml

```
DB_DBASE: cumple
DB_USER: intranet
DB_PASS: <PASSWORD_USUARIO_APP>
DB_ROOT_PW: <PASSWORD_ROOT_MARIADB>
```

Estas variables son consumidas tanto por el playbook de base de datos (creación de usuario, importación de datos) como por el playbook del servidor web (conexión de la app PHP a la base).

4. Playbooks

4.1 `playbooks/database.yaml` — Instalación y configuración de MariaDB

Qué hace:

1. Instala `mariadb-server`, `python3-pymysql` y `ufw`.
2. Inicia y habilita el servicio `mariadb`.
3. Configura `bind-address = 0.0.0.0` para permitir conexiones remotas (reinicia MariaDB si hubo cambios).
4. Abre el puerto `3306/tcp` en el firewall (UFW).
5. Verifica si la contraseña de `root` de MariaDB ya está inicializada; si no lo está, la establece (`DB_ROOT_PW`) usando las credenciales por defecto del socket unix.
6. Copia el script `cumples.sql` al servidor.
7. Importa la base de datos `cumples` desde ese script.
8. Crea el usuario `intranet` con privilegios `ALL` sobre la base `cumples`, accesible desde cualquier host (`%`).

Ejecución:

```
bash
ansible-playbook -i inventory playbooks/database.yaml
```

Resultado esperado:

- `PLAY RECAP` con `failed=0`.
 - En la primera ejecución: la tarea de inicialización de la contraseña de root se ejecuta (`changed`), la base `cumples` se importa y el usuario `intranet` se crea.
 - En ejecuciones posteriores: la tarea de verificación de contraseña de root falla intencionalmente y se ignora (`ignoring`) — esto es esperado y no indica un problema; el resto de las tareas deberían mostrarse como `ok` (idempotentes) sin cambios adicionales.
 - MariaDB queda escuchando en el puerto 3306, accesible remotamente, con la base `cumples` cargada.
-

4.2 `playbooks/webserver.yaml` — Instalación y configuración del servidor web

Qué hace:

1. Instala y habilita `firewalld`.
2. Abre los servicios `http` y `https` en el firewall.

3. Instala y habilita Apache (`httpd`).
4. Instala PHP, `php-mysqlnd`, `php-fpm` y `python3-libsemanage`.
5. Habilita el servicio `php-fpm`.
6. Despliega la aplicación (`cumple.j2`) como `/var/www/html/index.php`, con las variables de conexión a la base de datos ya reemplazadas.
7. Configura los booleanos de SELinux (`httpd_can_network_connect` y `httpd_can_network_connect_db`) para permitir que Apache/PHP se conecten a la base de datos remota.

Ejecución:

```
bash  
  
ansible-playbook -i inventory playbooks/webserver.yaml
```

Resultado esperado:

- `PLAY RECAP` con `failed=0`.
- Apache y `php-fpm` activos y habilitados al arranque.
- Puertos 80 y 443 accesibles desde el firewall.
- El archivo `/var/www/html/index.php` desplegado con las credenciales de conexión correctas.
- Al acceder por navegador a `http://<IP_DEL_SERVIDOR_WEB>`, la aplicación debe mostrar la lista de cumpleaños obtenida desde la base de datos remota (HTTP 200 OK, sin errores 500).

Orden de ejecución recomendado: correr primero `database.yaml` y luego `webserver.yaml`, ya que la aplicación web depende de que la base de datos y el usuario `intranet` ya existan.

5. Verificación de la aplicación

Pasos rápidos para confirmar que todo el flujo funciona (Apache → PHP → MariaDB):

```
bash  
  
# 1. Apache responde  
curl -I http://localhost/index.php  
  
# 2. Conectividad de red hacia la base de datos  
nc -zv <IP_DB> 3306  
  
# 3. Privilegios del usuario de la app  
mysql -u root -p -h 127.0.0.1 -e "SHOW GRANTS FOR 'intranet'@'%';"
```

Si todo está correcto, (`http://<IP_DEL_SERVIDOR_WEB>`) debe mostrar la tabla de cumpleaños sin errores.

6. Capturas de la aplicación funcionando

Ver capturas de pantalla de la aplicación en funcionamiento en el siguiente enlace:

<https://drive.google.com/EJEMPLO-REEMPLAZAR-CON-LINK-REAL>

7. Estructura del proyecto

```
taller-linux/  
├─ inventory  
├─ playbooks/  
|   ├─ database.yaml  
|   └─ webserver.yaml  
├─ tasks/  
|   └─ initialize_root.yaml  
├─ templates/  
|   └─ cumple.j2  
├─ files/  
|   └─ cumple.sql  
├─ vars/  
|   └─ database.yaml  
└─ README.md
```